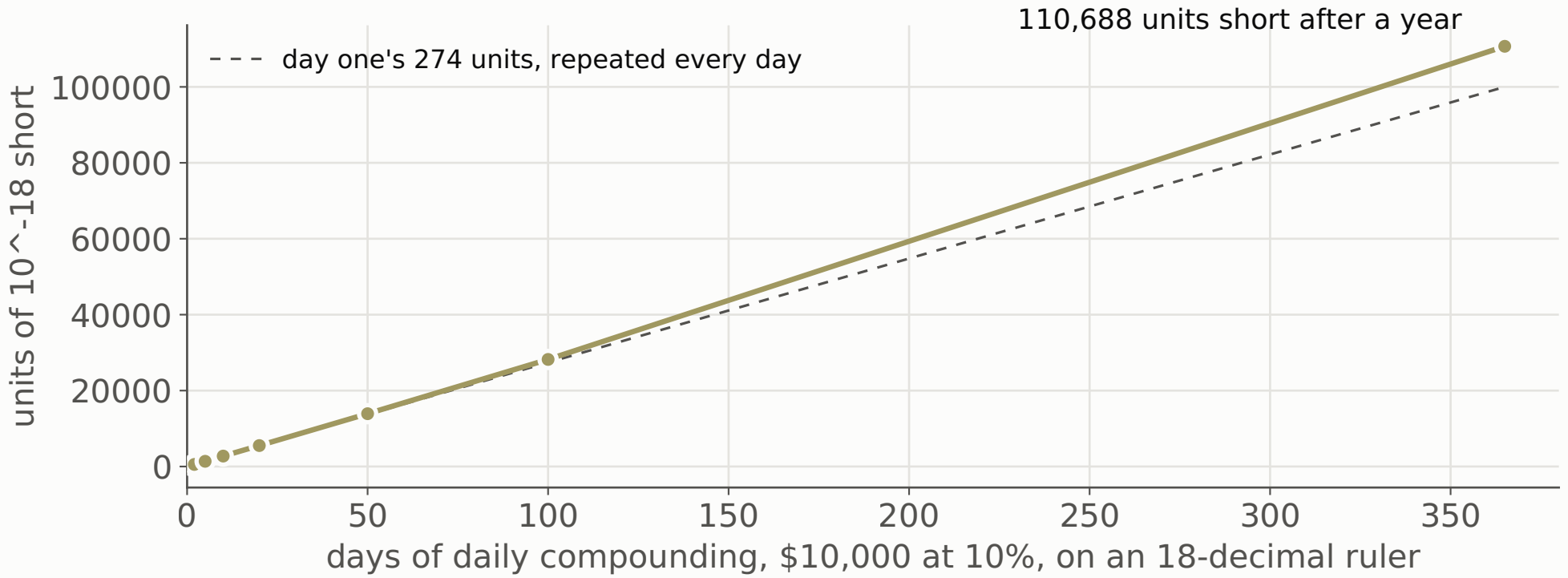
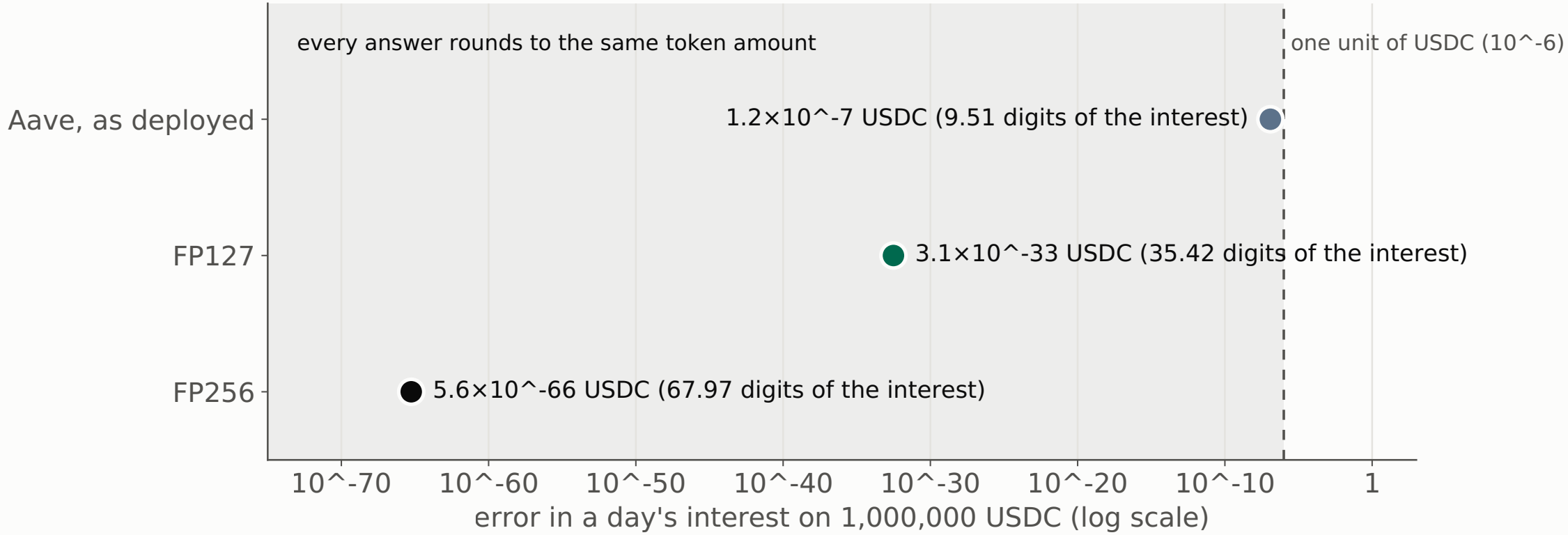
# Dust is Over



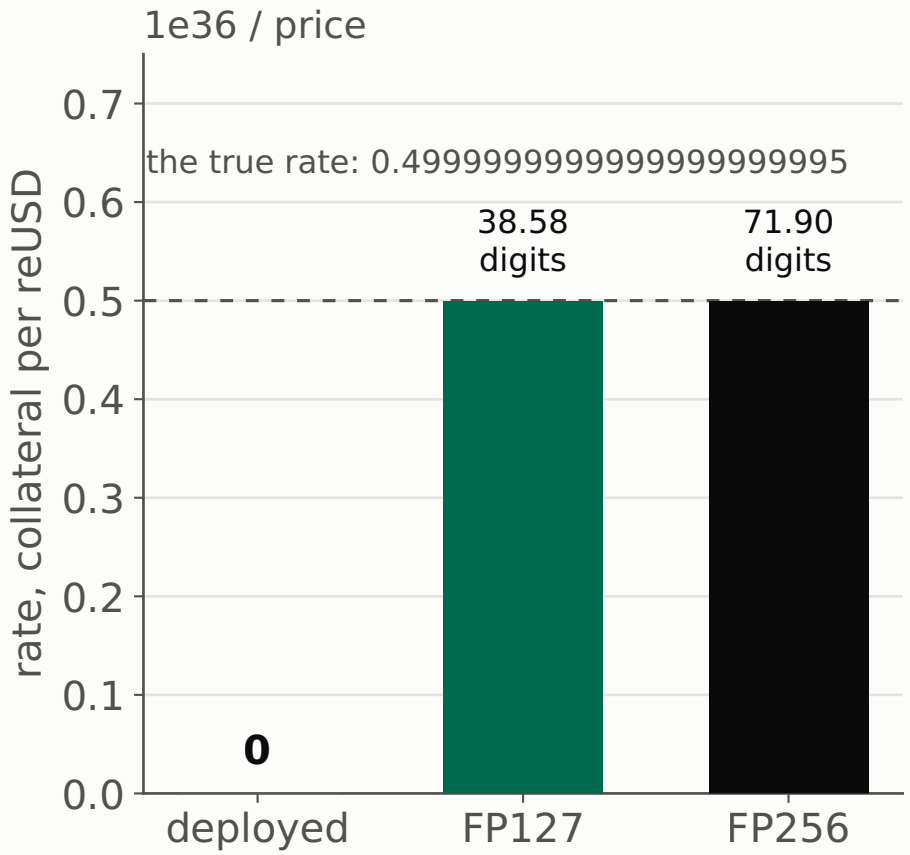

$10,000 at 10%, compounded daily on an 18-decimal ruler. docs/benchmarks/ladder.json: compound

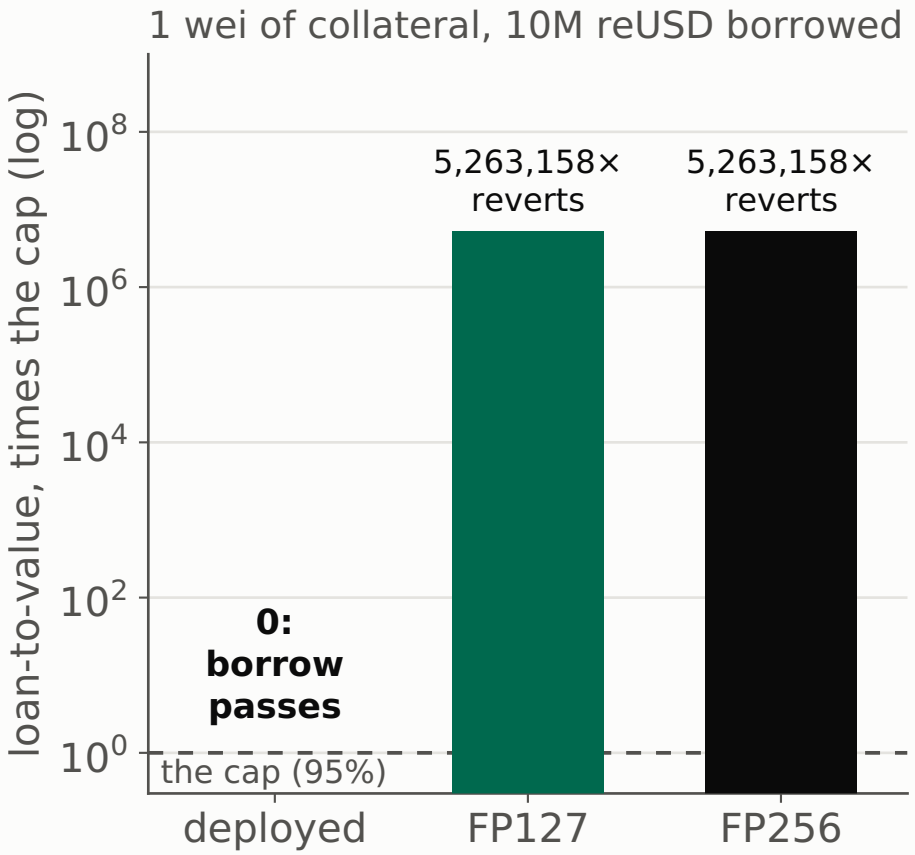
# One honest transaction




Aave v3 USDC borrow, 1M USDC at 100% utilisation, untouched a day; the deployed code against the formula at 200 digits. docs/benchmarks/accrual-deployed.json: aave-usdc-u100-borrow-1d

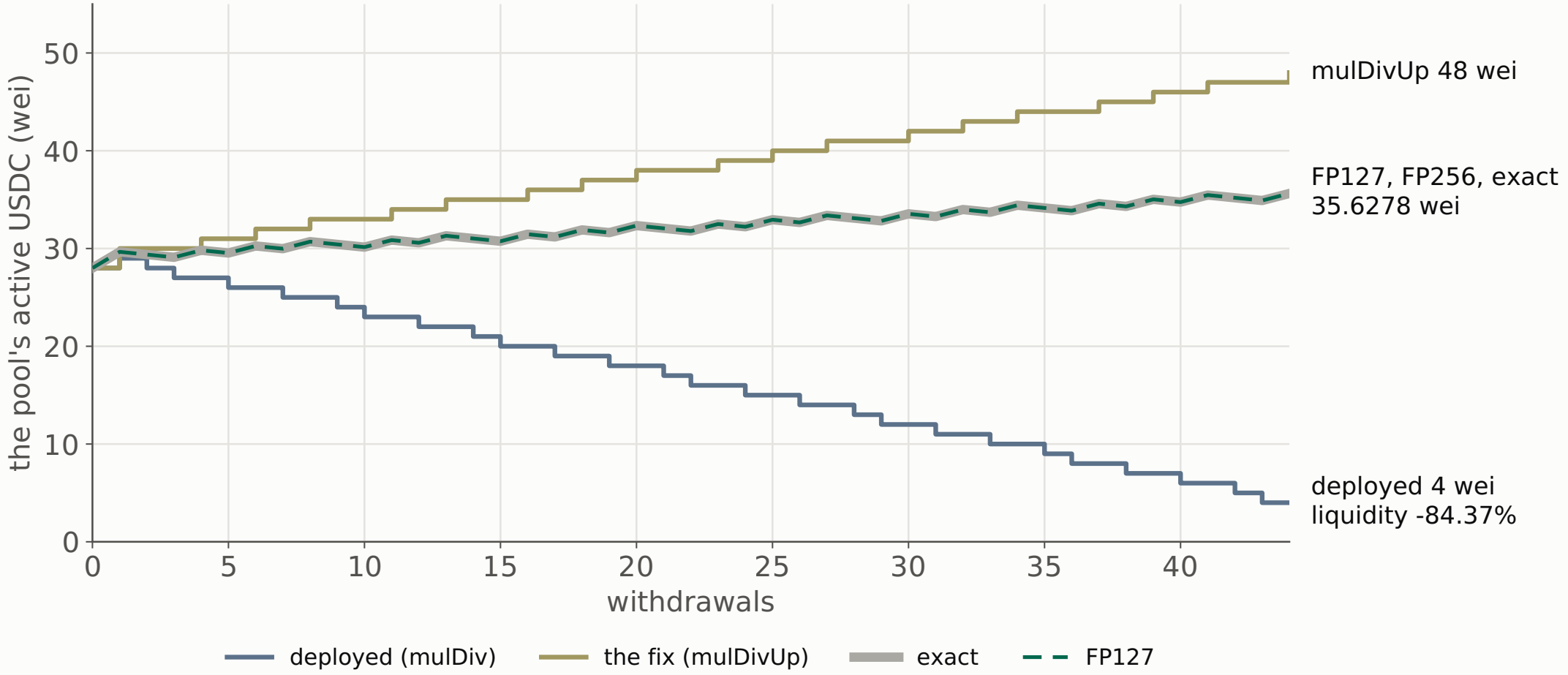
# Resupply, 26 June 2025



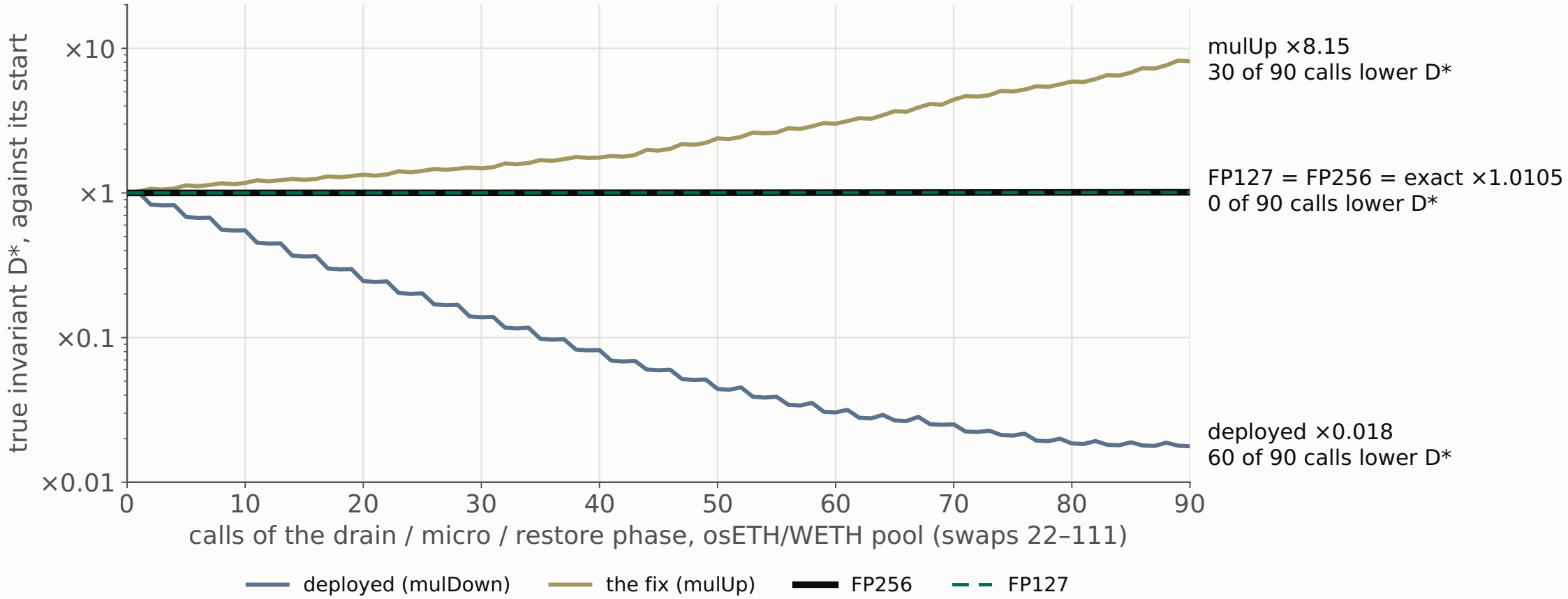

Tx 0xffbbd492…, block 22,785,461: the pinned integers, the same formula on each ruler. scripts/replays/resupply.json; docs/benchmarks/replay-resupply.md

# Bunni v2, 2 September 2025



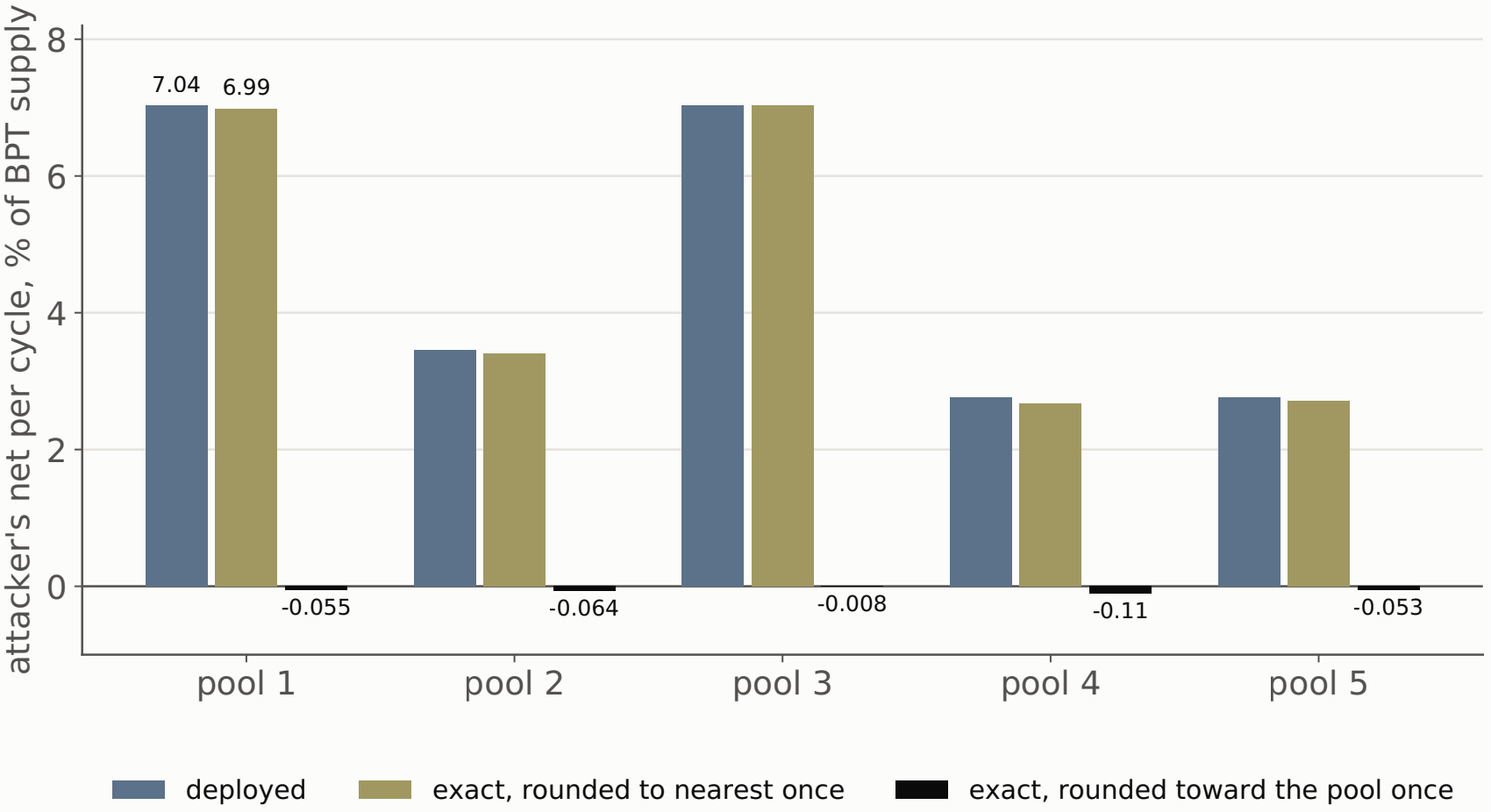

Tx 0x1c27c4d6..., block 23,273,098: the 44 withdrawals replayed from the trace, only the stored idle balance changed. scripts/replays/bunni.json (bunni.py replay); docs/benchmarks/replay-bunni.md

# Balancer V2, 3 November 2025



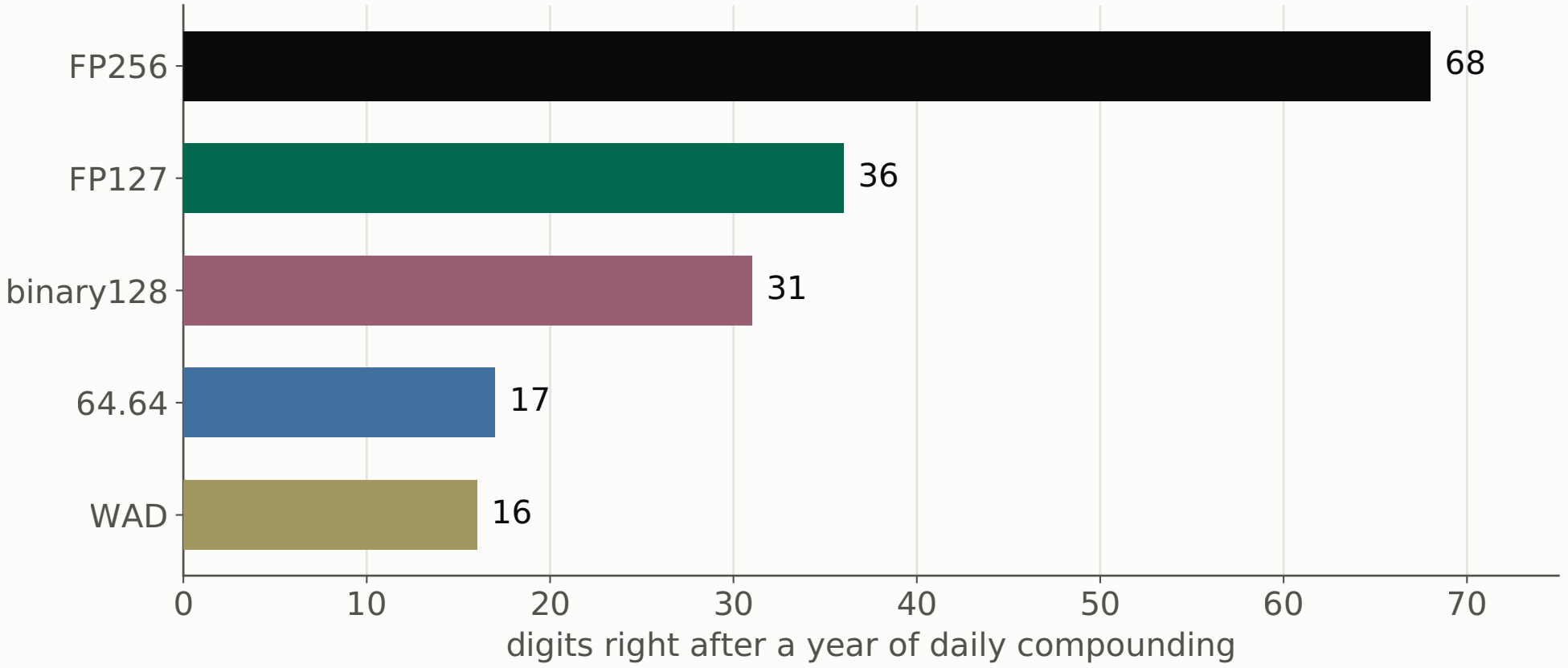

Tx 0x6ed07db1…, block 23,717,397: every call priced on its own on-chain balances; D* solved exactly. scripts/replays/balancer.json (score.osETH); docs/benchmarks/replay-balancer.md

# Balancer V1, 31 August 2026



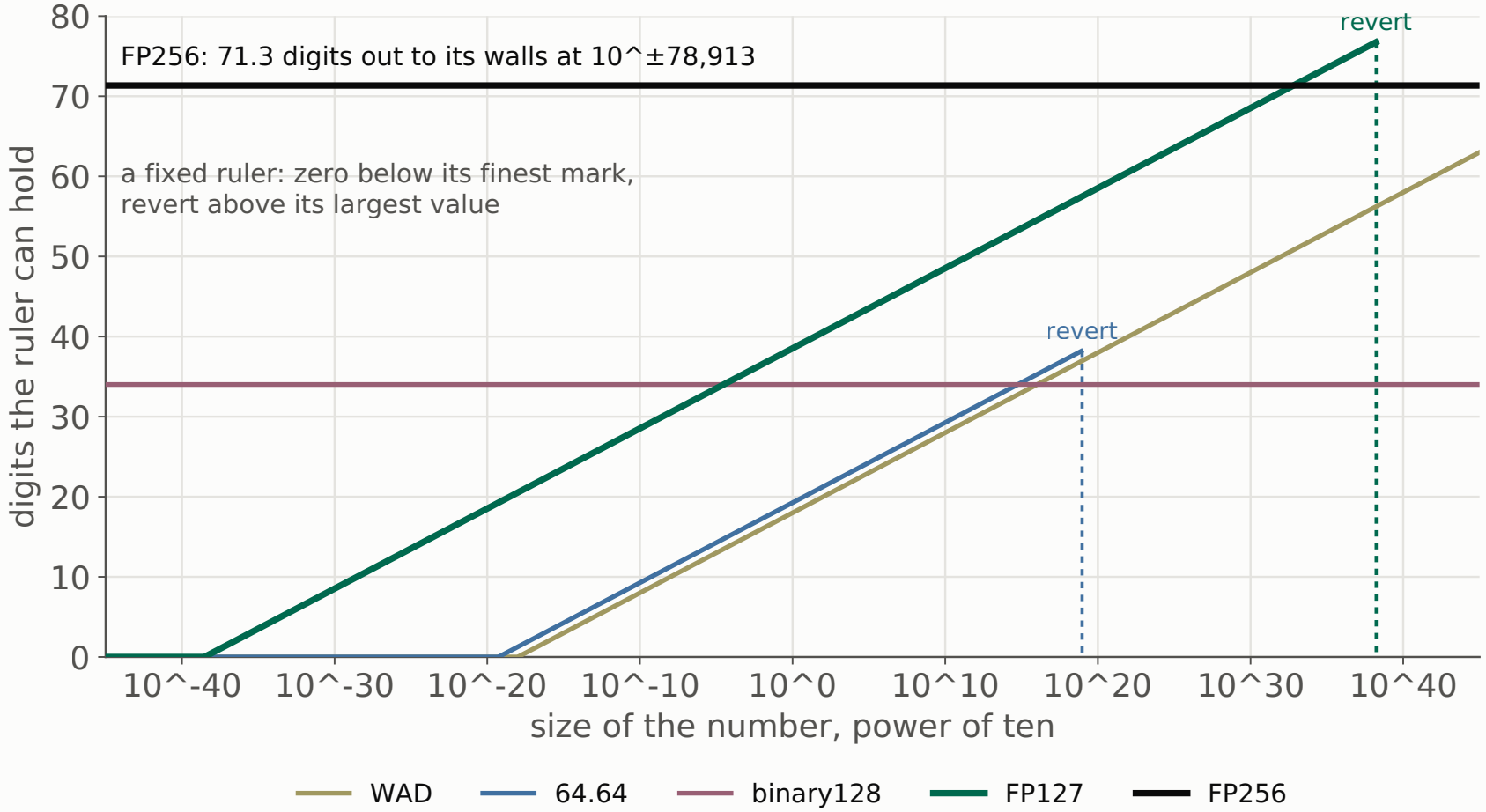

Five pools; from each pool's first join at 3 units, one join/exit cycle re-tuned to each rule at 200 digits. docs/benchmarks/replay-balancer-v1.md: Precision or direction?

# Goal one: round once



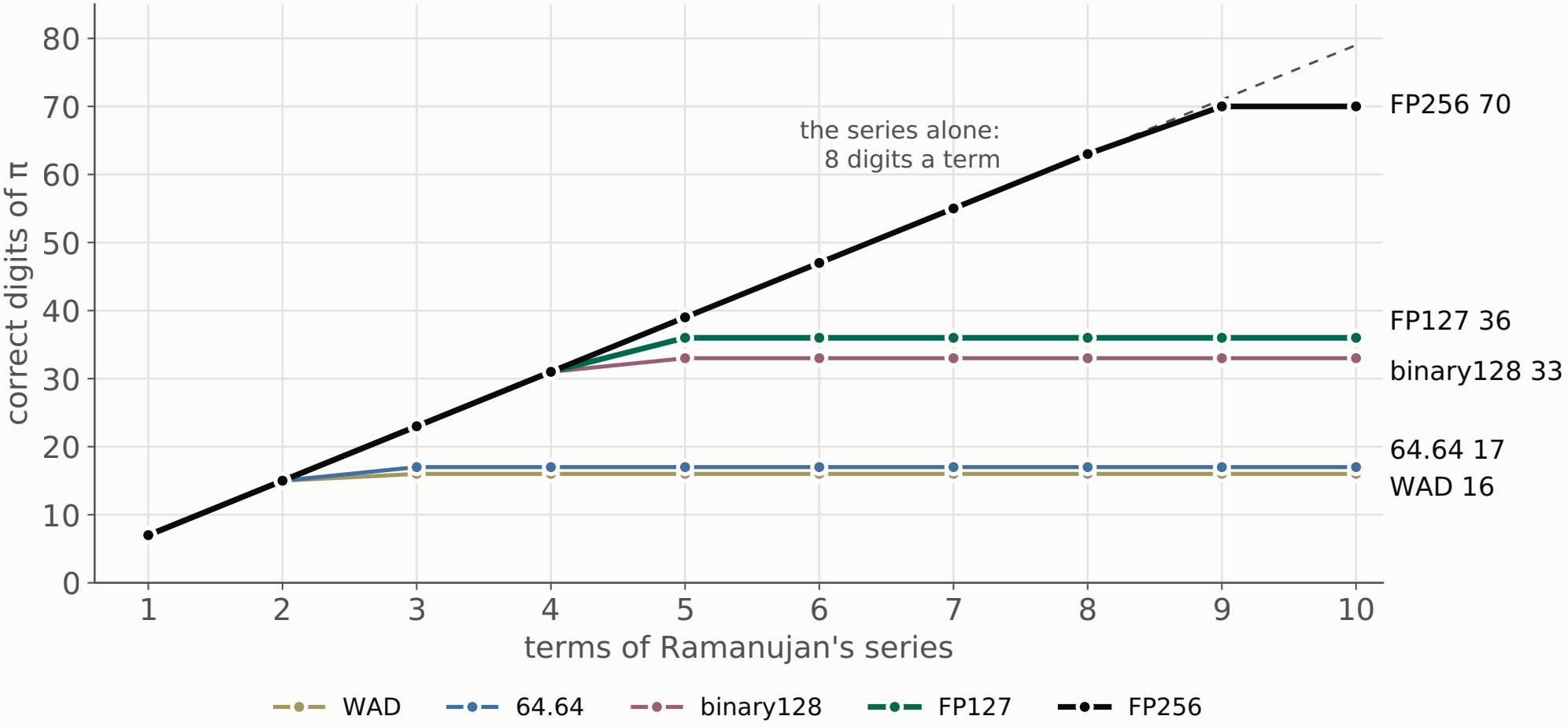

A year of daily compounding, $10,000 at 10%. docs/benchmarks/ladder.json. WAD = an 18-decimal library; 64.64 and binary128 = the two ABDK formats.

# Goal two: never run out of ruler



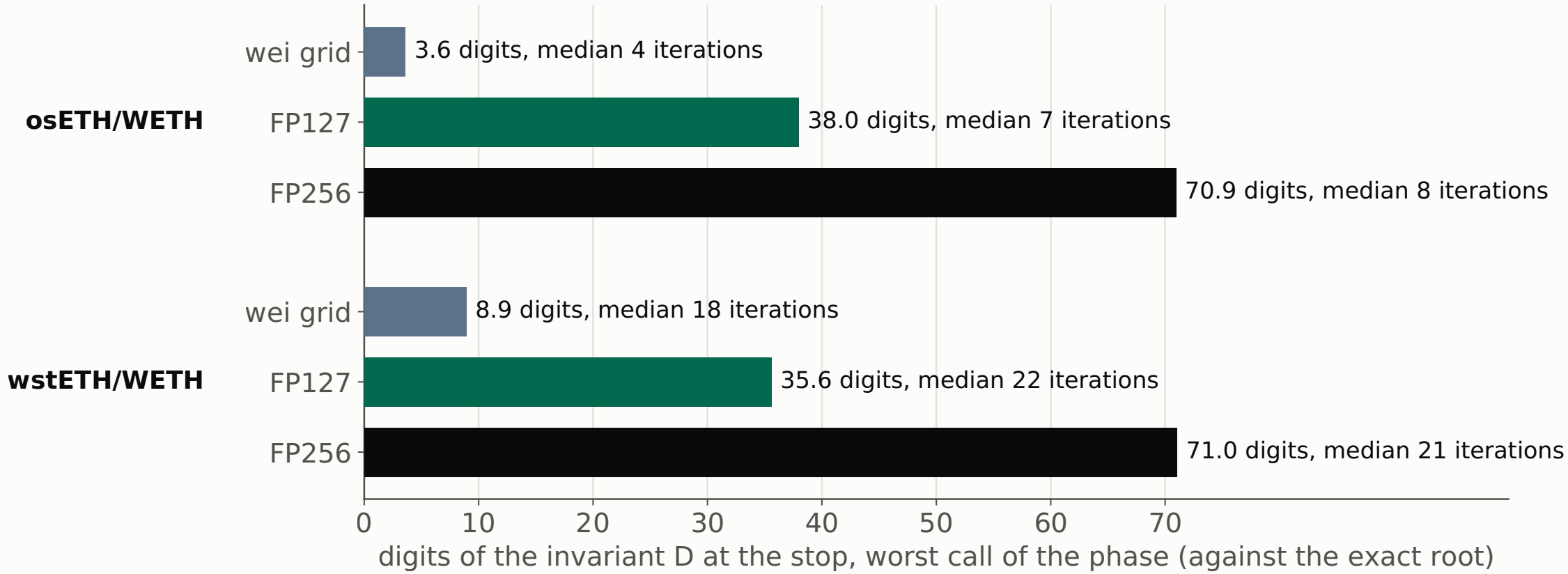

Exact properties of each format, computed in scripts/talk/deck.py.

# Ramanujan



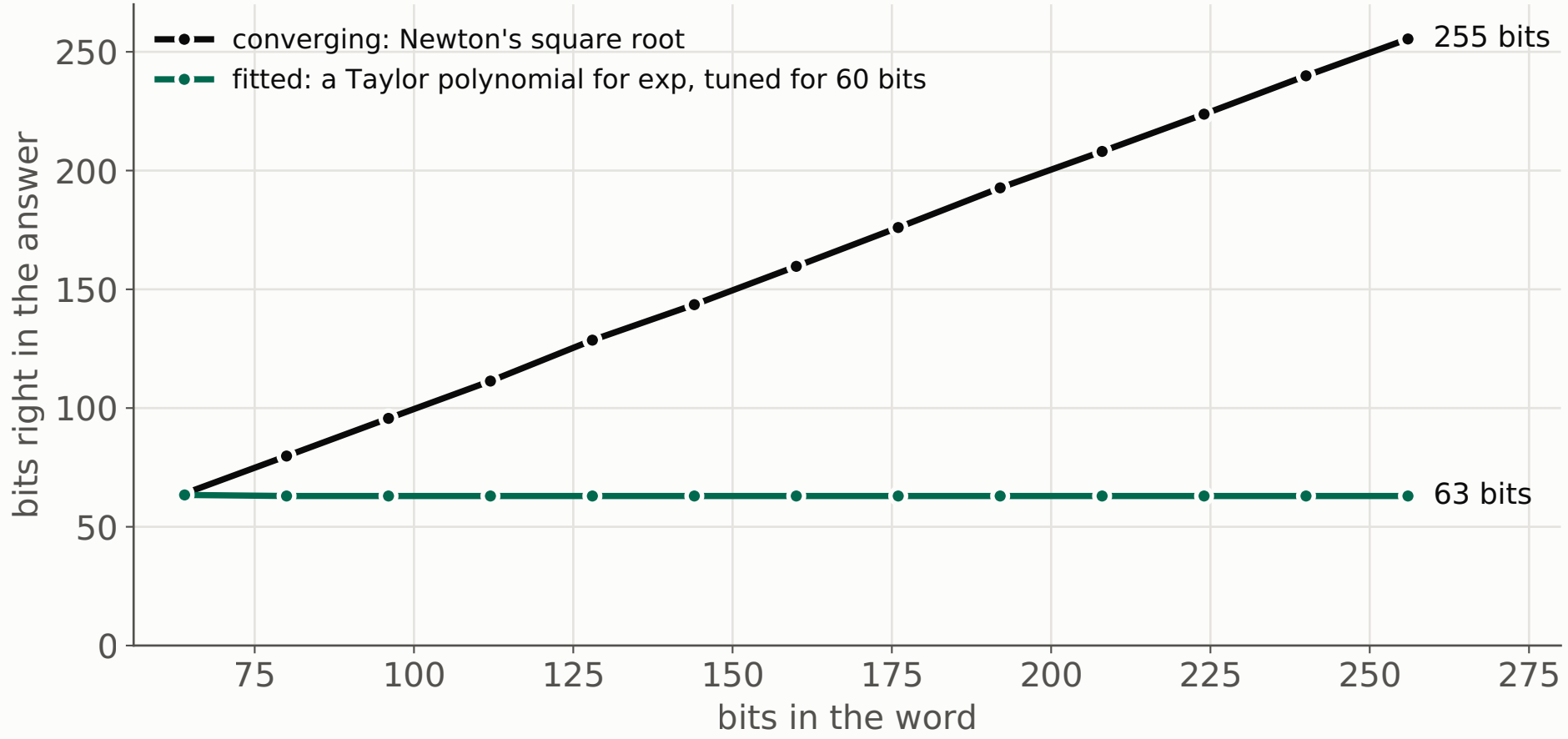

Ramanujan's series through each library's own sqrt, mul and div. docs/benchmarks/pi.md

# Balancer's Newton solve, three rulers



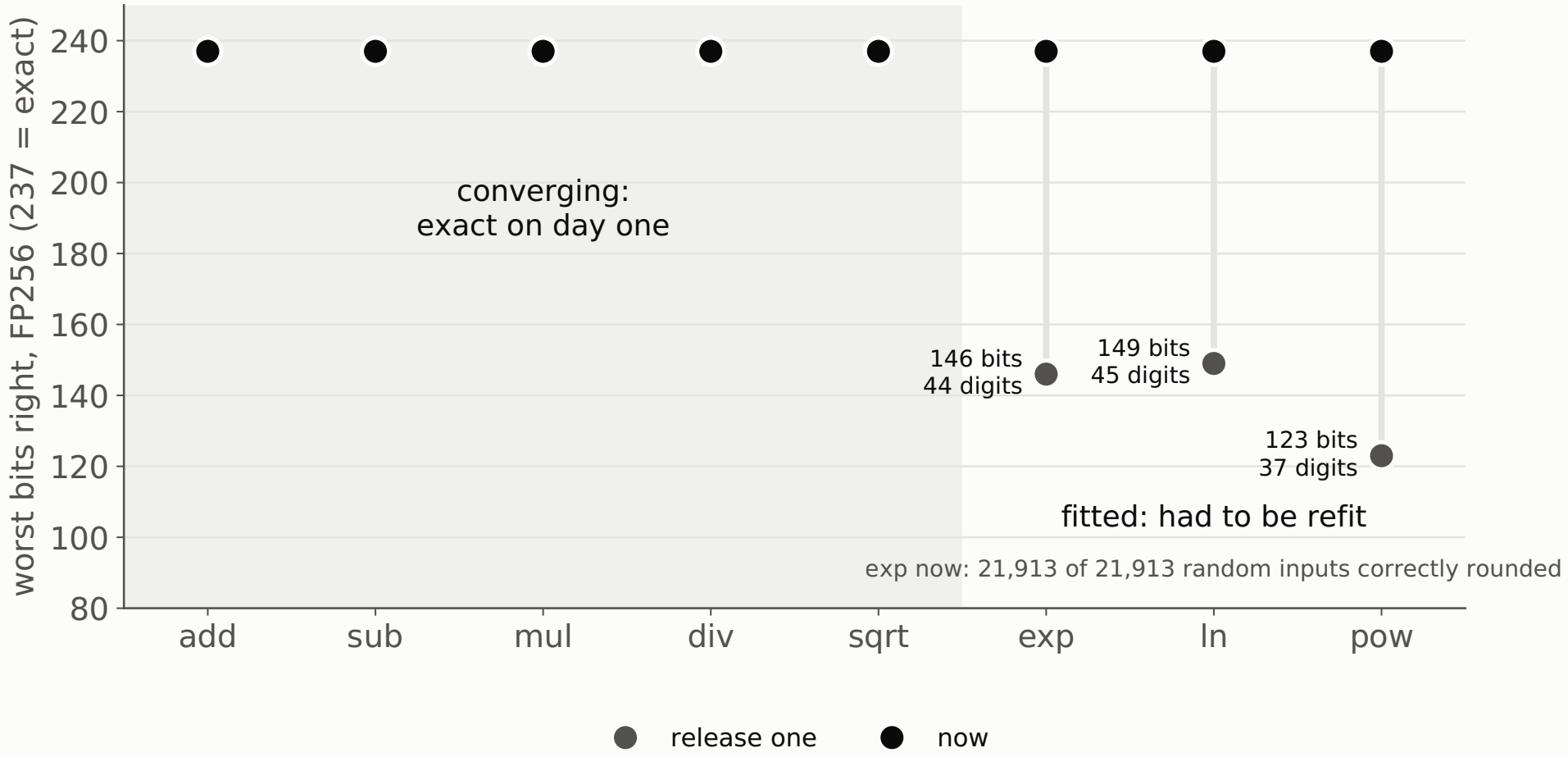

StableMath's invariant iteration on each ruler, residual against the exact root at 200 digits. scripts/replays/balancer.json (score.*.convergence); docs/benchmarks/replay-balancer.md: Newton convergence

# Converge, or fit



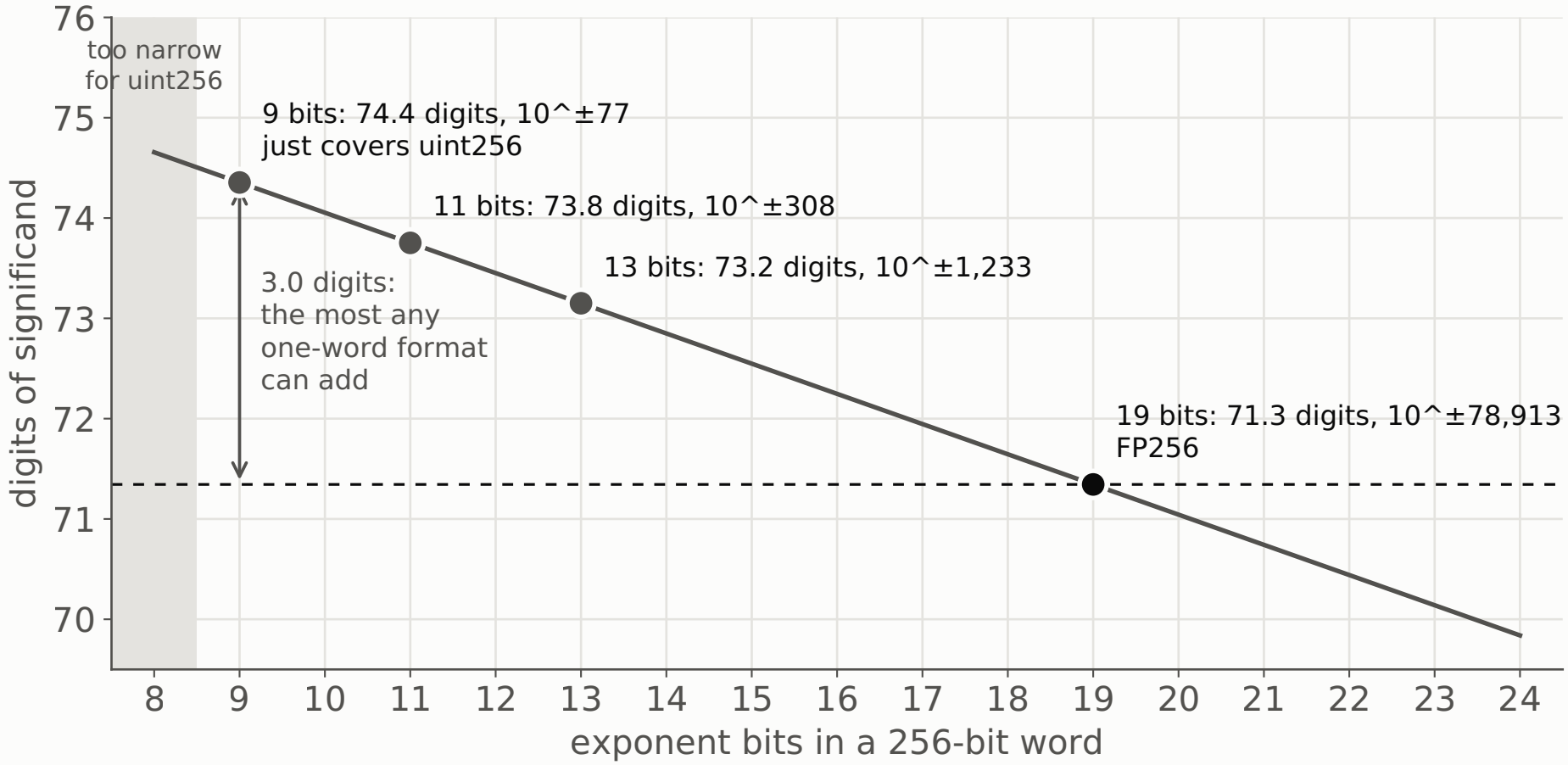

Computed with mpmath at each width: Newton's sqrt(2), and a Taylor polynomial for exp tuned for 60 bits (the shortest within 2^-60 on [0, ln 2]), at x = ln 2. scripts/talk/deck.py: converge_vs_fit

# Release one, and now



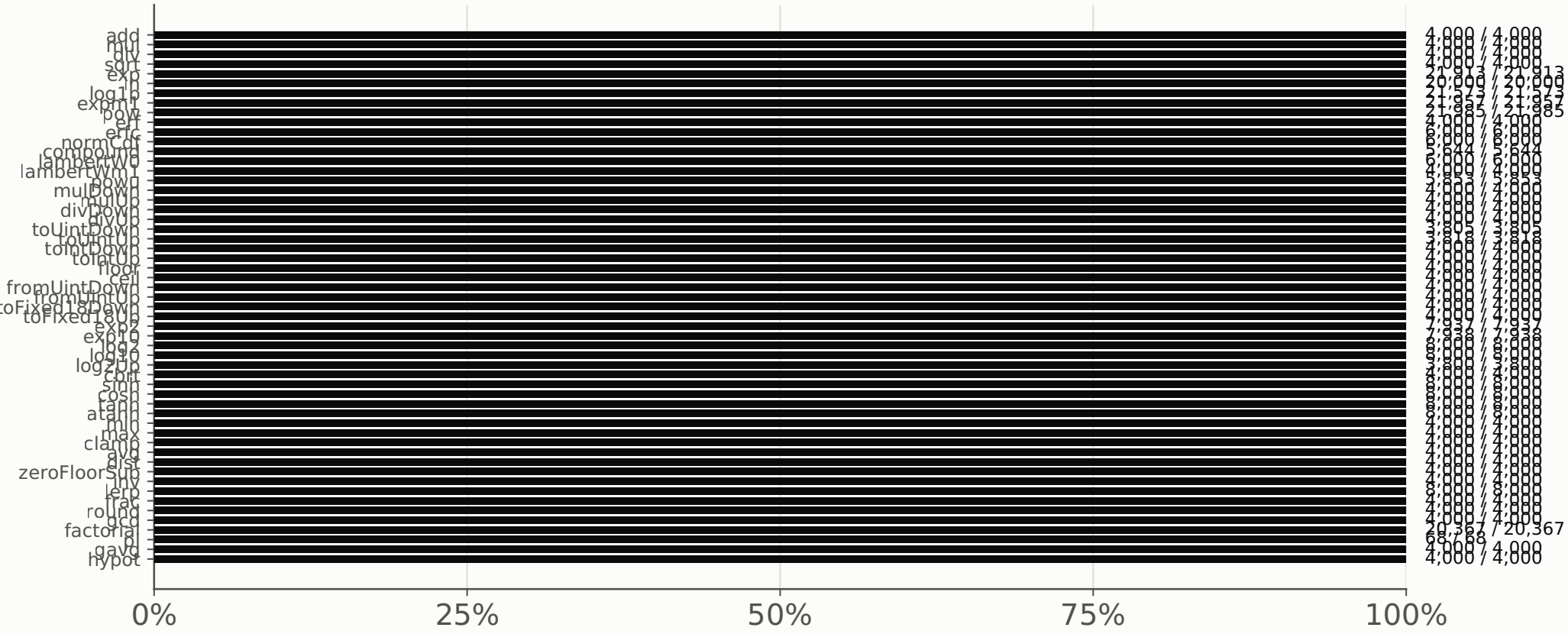

Worst bits over the committed vectors. Release one: contracts/deployments/FP256.md; now: docs/fp256/precision.json (precision.md) and docs/benchmarks/sweep.json

# The format ceiling




A 256-bit word: sign, e exponent bits, 256 − e significand bits with the hidden bit; range 2^±2^(e−1). Computed in scripts/talk/deck.py: plot_format_ceiling

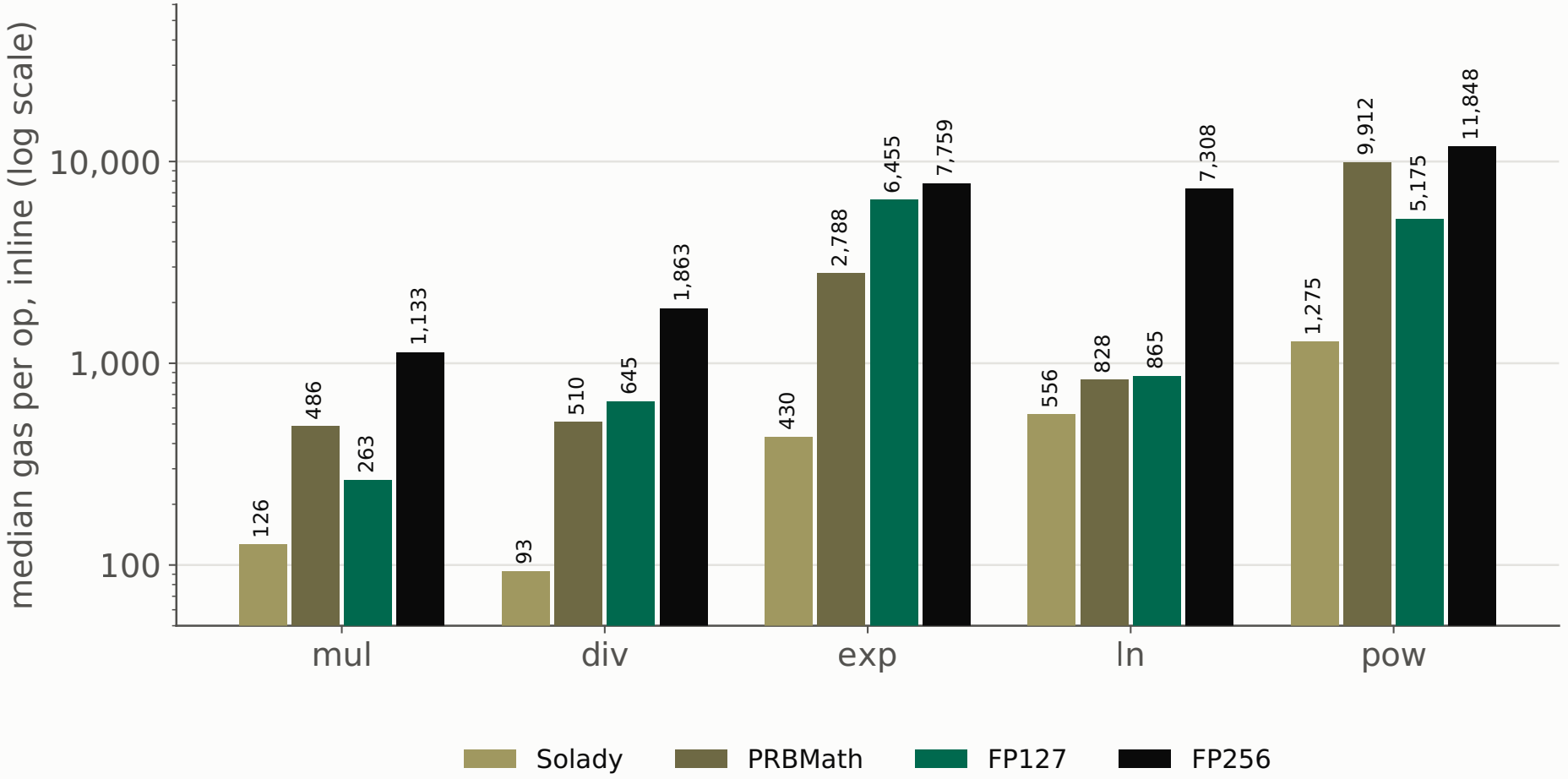
# The accuracy ceiling




FP256: share of random inputs that return the correctly rounded word
and 97,206 more forced through the second stage of 20 ops: 0 misrounded

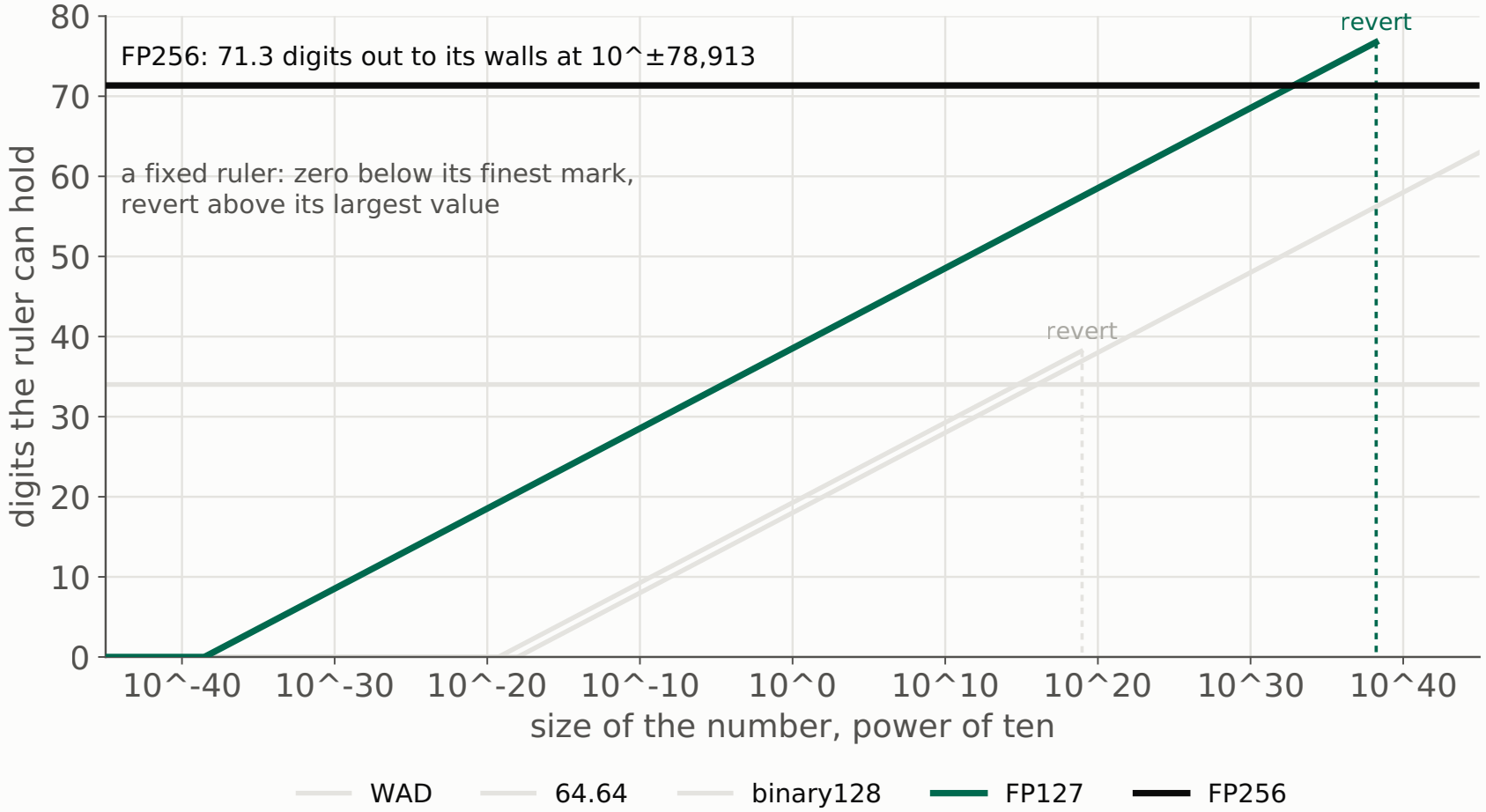
Seeded random inputs against the correctly rounded word (300 digits, ties away from zero). docs/benchmarks/sweep.json (sweep.md)

# What it costs



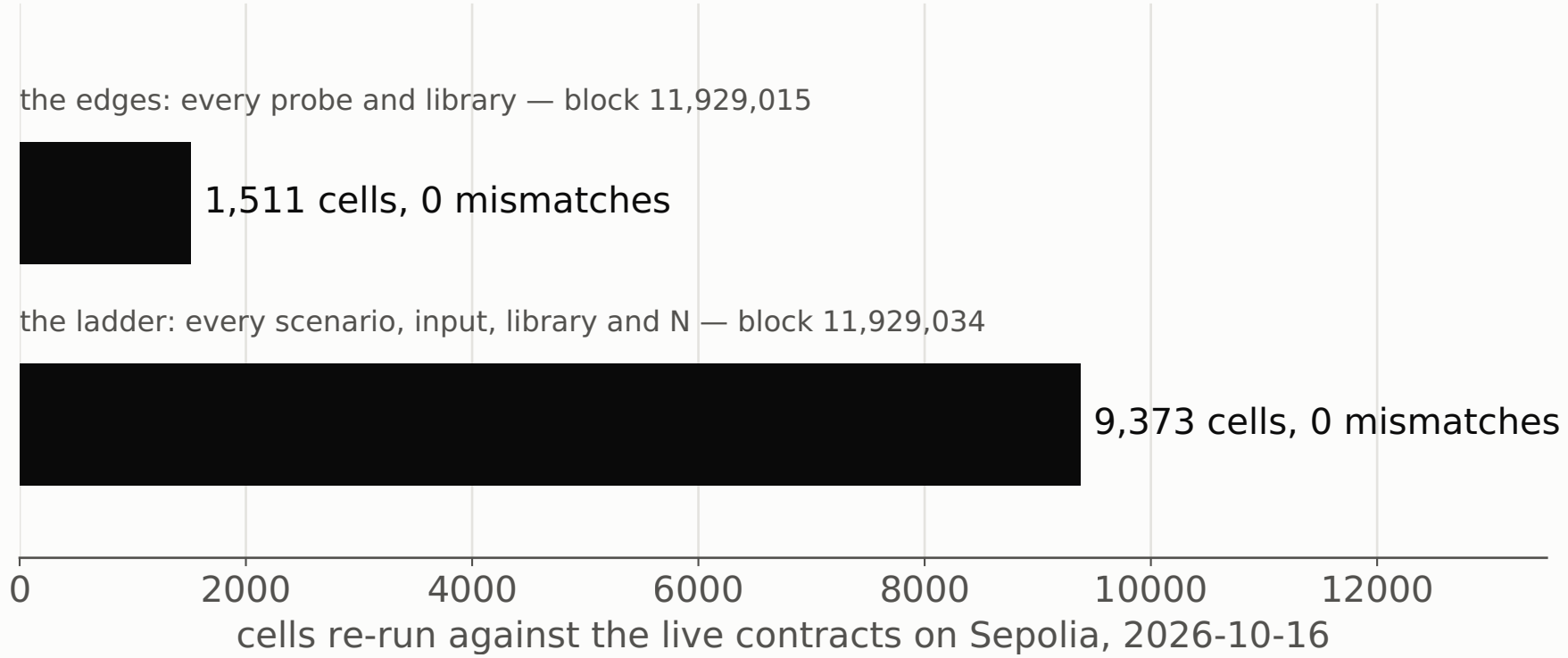

Median gas over each op's input ladder, inline libraries. docs/benchmarks/gas.md (Solady, PRBMath SD59x18, FP127Lib); docs/benchmarks/gas256.md (FP256Lib)

# Both goals, met




Exact properties of each format, computed in scripts/talk/deck.py.

# Checked on chain




contracts/deployments/RELEASE.json

# Use FP127 and FP256

## FP127

0x619176d72D7a12390F8970802943D344c0fd3fE2

## FP256

0xA3C8C8700465F9e20Ff9dfA64066c2EF693Aa6c9

FP256X 0x175246B2…40d0 · FP256X2 0xE655bB8b…c697 · FP256X3 0x59D51481…2383

blockulator.com · github.com/securedataresearch/blockulator